

AUTH SYSTEM

Complete Developer Documentation

DOCUMENT 3 OF 4

Security & Secrets Management

1. The Real Story — What Happens Without Secrets Management

This is not a hypothetical scenario. This exact sequence of events destroys real companies every week:

Step	What Happens
1	Developer hardcodes the database password directly inside a Python or JavaScript source file
2	Developer commits the code and pushes to a public GitHub repository
3	Automated bots scan GitHub continuously — within minutes they find the exposed password
4	The bot sends the credential to criminals who immediately access the live database
5	All user data is downloaded or destroyed. The company faces legal action and ransom demands
6	Even after deleting the file, the password lives in Git history forever unless every commit is rewritten

This Is Why .env Files Exist

Environment files were invented specifically to solve this problem. The code lives in Git. The secrets live only on the machine that needs them. They are connected at runtime — never at commit time.

2. Environment Variables — The Core Concept

An environment variable is a named value stored on your operating system — not in your code files. Your program reads it at startup. Think of it as a sticky note on your computer that only your computer can see.

2.1 Same Code, Different Behaviour on Each Machine

Variable Name	Your Laptop (Development)	Live Server (Production)
APP_ENV	development	production
DEBUG	true	false — NEVER true on live server
DATABASE_URL	localhost:5432/testdb	prod-server.internal:5432/livedb
SENDGRID_API_KEY	fake_key (no real emails sent)	SG.real_key (real emails sent to users)
SECRET_KEY	short-dev-key-acceptable	64-character cryptographic random key
COOKIE_SECURE	false (HTTP fine locally)	true (HTTPS mandatory)

The config.py file reads every variable from the environment automatically:

```
# app/core/config.py — simplified to show the concept
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    # Pydantic reads DATABASE_URL from the environment automatically
    # On your laptop: returns 'postgresql://localhost/testdb'
    # On live server: returns 'postgresql://prod-server/livedb'
    # The CODE is identical on both machines
    DATABASE_URL: str
    SECRET_KEY: str
    DEBUG: bool = False

    # This tells Pydantic to also read from a .env file
    model_config = SettingsConfigDict(env_file='.env')

# If a required variable is MISSING the app refuses to start
# This prevents running with an incomplete configuration
```

3. The Three Files You Must Understand

There are exactly three environment-related files in this project. Knowing which ones are safe to commit to GitHub is essential:

File	Contains	In .gitignore?	Safe for GitHub?
.env	Your REAL secrets	YES — always	NEVER — not ever
.env.example	Empty template only	NO	YES — safe to commit
.env.local (Next.js)	Frontend local settings	YES — automatically	NEVER

3.1 The .env File — Your Private Secrets (Never Share This)

```
# backend-api/.env
# _____
# THIS FILE NEVER GOES TO GITHUB. EVER.
# Every developer on the team has their OWN separate copy.
# The live server has its own completely separate version.
# _____

APP_ENV=development
DEBUG=true

# Generate with: python -c "import secrets; print(secrets.token_urlsafe(64))"
SECRET_KEY=kR9xP2mV8nQ4jL6tY1wZ5bA3cE7hF0dGxR9xP2mV8nQ4jL6tY1wZ5bA3
OTP_HMAC_KEY=uJ8mK2pN4rQ6sT0vW3xY7zA1bC5dE9fH

# Your local PostgreSQL running in Docker
DATABASE_URL=postgresql+asyncpg://authuser:devpassword@localhost:5432/authdb

# Your local Redis running in Docker
REDIS_URL=redis://localhost:6379/0

# Fake values — real services not needed for local testing
SENDGRID_API_KEY=SG.fake_local_key_not_real
TWILIO_AUTH_TOKEN=fake_token_local

# Disable to avoid flooding support inbox during development
ERROR_NOTIFICATIONS_ENABLED=false
COOKIE_SECURE=false
```

3.2 The .env.example File — Safe Template for GitHub

```
# backend-api/.env.example
# _____
```

```
# SAFE TO COMMIT — this file contains no real secrets
# New developer setup: cp .env.example .env
# Then fill in your own values
# _____

APP_ENV=development
DEBUG=true

# Generate: python -c "import secrets; print(secrets.token_urlsafe(64))"
SECRET_KEY=replace-with-your-own-64-character-random-string
OTP_HMAC_KEY=replace-with-your-own-32-character-random-string

DATABASE_URL=postgresql+asyncpg://USERNAME:PASSWORD@localhost:5432/authdb
REDIS_URL=redis://localhost:6379/0

# Get from twilio.com/console (free trial available)
TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=your_twilio_auth_token_here
TWILIO_FROM_NUMBER=+1xxxxxxxxx

# Get from app.sendgrid.com (free tier available)
SENDGRID_API_KEY=SG.xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
EMAIL_FROM=noreply@yourdomain.com

# Get from console.cloud.google.com
GOOGLE_CLIENT_ID=xxxxxxxxxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxxxxxx

SUPPORT_EMAIL=support@yourdomain.com
ERROR_NOTIFICATIONS_ENABLED=false
```

4. The .gitignore File — Protecting Your Secrets

The .gitignore file tells Git which files to permanently ignore. Files listed here are never included in commits and never sent to GitHub — no matter what.

```
# auth-system/.gitignore
# _____
# THE MOST IMPORTANT LINES:
.env
.env.*
!.env.example

# Python virtual environment
venv/
env/
.venv/
__pycache__/
*.pyc
*.pyo
.pytest_cache/
htmlcov/
coverage.xml
.mypy_cache/
.ruff_cache/

# Node.js
node_modules/
.next/
out/
build/

# IDE files (optional — your team may want to keep these)
.idea/
.vscode/
*.swp

# Operating system
.DS_Store
Thumbs.db

# Certificate and key files — extra protection
*.pem
*.key
*.cert
secrets/
```

4.1 Verify .gitignore Is Working

```
# Run from the project root — .env must NOT appear
git status
```

```
# Check if a specific file is being ignored
git check-ignore -v backend-api/.env
# Safe output: backend-api/.gitignore:2:.env  backend-api/.env

# DANGER: if this returns nothing — the file is NOT ignored
# Add it to .gitignore immediately before making any commit

# Check if .env was EVER committed by mistake
git log --all -- '**/.env'
# Safe output: (nothing — no results means never committed)
```

Emergency — Accidentally Committed Secrets

If you committed a .env file or real password: (1) Immediately rotate every exposed credential — change passwords, regenerate API keys. Treat them as fully compromised. (2) Add the file to .gitignore. (3) Run: `git rm --cached backend-api/.env` then commit. (4) Contact every affected service provider to invalidate old keys. The old commit still exists in history but the credentials are now invalid.

5. The Three Environments — Local, Staging, Production

Every professional application has at least three separate environments. Each runs independently with its own database, its own secrets, and its own configuration.

	Local (Your Laptop)	Staging (Test Server)	Production (Live Server)
Purpose	Write and test code	Test before going live	Serve real users
Data	Fake test data only	Realistic fake data	Real user accounts
A mistake means	Nothing — only you affected	Nothing — no real users	Critical — users impacted
DEBUG	true	false	false — NEVER true
Secrets location	.env file on your laptop	Server environment variables	Secrets manager (AWS/Vault)
HTTPS needed	No — localhost is fine	Yes	Yes — always required
Developer access	Only yourself	Dev team (read-only DB)	Nobody — automated only

5.1 Why Developers Must Not Access Production Secrets

- A developer who can access production can accidentally break or expose live user data
- If a developer's laptop is stolen the attacker must not be able to reach the live database
- Compliance regulations like GDPR and PCI-DSS often legally require this separation
- In a small team today — use a secrets manager like Doppler or AWS Secrets Manager from day one
- All development and all testing must happen against local or staging data — never production

The Golden Rule of Secrets

The only entity that should ever know a production password is the automated deployment system. Human developers should need zero production secrets to do their jobs. Everything they need for development lives in their local .env file.

6. How to Set Secrets on Each Environment

6.1 Local — .env File

```
# Each developer creates their own .env from the template
cp backend-api/.env.example backend-api/.env

# Edit it with personal test credentials
# FastAPI reads it automatically via Pydantic Settings:
# model_config = SettingsConfigDict(env_file='.env')
```

6.2 Staging and Production — Server Environment Variables

```
# Set on the Linux server directly (done once by DevOps)
export DATABASE_URL='postgresql+asyncpg://user:pass@server/db'
export SECRET_KEY='production-64-char-random-key'

# Or via a file with restricted permissions
chmod 600 /opt/auth-system/backend-api/.env
chown appuser:appgroup /opt/auth-system/backend-api/.env

# Via Docker Compose (use --env-file or environment: block)
# NEVER put real values in docker-compose.yml itself
# Always reference variables: SECRET_KEY=${SECRET_KEY}
```

6.3 CI/CD Pipeline — GitHub Secrets

The GitHub Actions pipeline needs secrets for deployment. Set them in GitHub — they are encrypted and never appear in any log:

1. Go to your GitHub repository
2. Click Settings > Secrets and variables > Actions
3. Click New repository secret for each production credential
4. In your workflow file reference them as: `${{ secrets.SECRET_NAME }}`

```
# .github/workflows/ci.yml — referencing secrets safely
- name: Deploy to server
  env:
    DATABASE_URL: ${ secrets.PROD_DATABASE_URL }
    SECRET_KEY:   ${ secrets.PROD_SECRET_KEY }
  run: ./deploy.sh
# Secrets appear as *** in all logs — completely hidden
```

7. New Developer Onboarding — Step by Step

How to get a new team member contributing code without ever touching any production credential:

5. New developer clones the repository: `git clone https://github.com/yourorg/auth-system.git`
6. Copy the environment template: `cp backend-api/.env.example backend-api/.env`
7. Create a FREE Twilio trial account at [twilio.com](https://www.twilio.com) and copy the test credentials into `.env`
8. Create a FREE SendGrid account at sendgrid.com and copy the API key into `.env`
9. Generate personal SECRET_KEY: `python -c "import secrets; print(secrets.token_urlsafe(64))"`
10. Start the stack: `docker-compose up --build`
11. Run migrations: `docker-compose exec backend alembic upgrade head`
12. Open `http://localhost:3000` — everything works with their personal fake credentials
13. They can build features, run tests, and submit pull requests — zero production access needed

8. Complete Secrets Security Checklist

Verify every item before any deployment and when onboarding new developers:

	Checklist Item
[]	backend-api/.env is listed in .gitignore
[]	frontend-app/.env.local is listed in .gitignore
[]	git status does NOT show any .env file
[]	git log --all -- '**/.env' returns no results
[]	Production SECRET_KEY is minimum 64 characters, randomly generated
[]	Different SECRET_KEY values for development, staging, and production
[]	COOKIE_SECURE=true in production
[]	DEBUG=false in production
[]	ERROR_NOTIFICATIONS_ENABLED=true in production
[]	No developer has SSH access to the production database server
[]	All API keys are environment-specific (separate Twilio/SendGrid accounts)
[]	GitHub repository Secrets are set for all CI/CD pipeline variables
[]	Every new developer has read this document before writing any code

Final Word

Treat every password and API key like a physical house key. You would not photocopy your house key and put it in a public library. Apply the same logic to digital secrets. When in doubt —

rotate the credential. It takes five minutes and completely eliminates the risk.